OPERATING SYSTEMS (UE24CS242B)

Unit 1 – CPU Scheduling Algorithms – FCFS and SJF

Scheduling Algorithms

Scheduling algorithms decide which process gets the CPU and for how long. In this lecture, we study:

- **First-Come, First-Served (FCFS)**
- **Shortest-Job-First (SJF)** and its preemptive form

First-Come, First-Served (FCFS) Scheduling

FCFS is the simplest CPU scheduling algorithm.

Characteristics

- Processes are executed in the order they arrive
- Uses a FIFO queue
- Non-preemptive algorithm
- Once CPU is allocated, the process runs until completion or I/O wait

FCFS Example – Case 1

Suppose the processes arrive in the order: **P_1, P_2, P_3**

Process	Burst Time
P1	24
P2	3
P3	3

Waiting Time

- Waiting time of $P_1 = 0$
- Waiting time of $P_2 = 24$
- Waiting time of $P_3 = 27$

Average Waiting Time

$$(0 + 24 + 27)/3 = 17$$

FCFS Example – Case 2

Suppose the processes arrive in the order: **P_2, P_3, P_1**

Waiting Time

- Waiting time of $P_2 = 0$
- Waiting time of $P_3 = 3$
- Waiting time of $P_1 = 6$

Average Waiting Time

$$(6 + 0 + 3)/3 = 3$$

Observation

- FCFS **does not guarantee minimum average waiting time**
- Performance varies greatly when CPU burst times differ significantly

Drawbacks of FCFS Scheduling

Convoy Effect

- Short processes get stuck **behind a long process**
- Example:
    - One CPU-bound process
    - Many I/O-bound processes

Additional Issues

- FCFS is **non-preemptive**
- A process can occupy the CPU for a long time
- Poor choice for **time-sharing systems**
- Not fair when responsiveness is important

Shortest-Job-First (SJF) Scheduling

SJF associates each process with the **length of its next CPU burst**.

Scheduling Rule

- Process with the **shortest CPU burst** is scheduled next

Key Properties

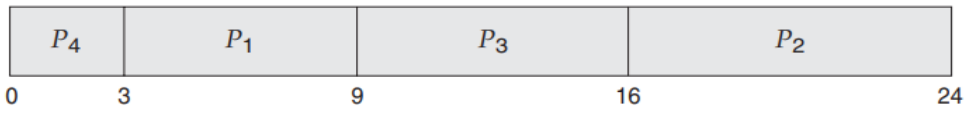
- Optimal scheduling algorithm
- Produces minimum average waiting time
- Can be:
    - Non-preemptive (SJF)
    - Preemptive (Shortest-Remaining-Time-First)

Main Challenge

- Exact length of next CPU burst is **not known in advance**
- Must be **predicted**

Example of SJF Scheduling

Process	Burst Time
P1	6
P2	8
P3	7
P4	3



Average Waiting Time (SJF)

$$(3 + 16 + 9 + 0)/4 = 7$$

Comparison with FCFS

If FCFS is used:

$$(0 + 6 + 14 + 21)/4 = 10.25$$

Observation

- SJF gives better (lower) average waiting time than FCFS

Determining Length of Next CPU Burst

Since actual burst time is unknown, OS predicts it using **exponential averaging**.

Formula

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

Where:

- τ_{n+1}→ predicted next CPU burst
- t_n→ actual last CPU burst
- τ_n→ previous prediction
- α→ weighting factor ($0 \leq \alpha \leq 1$)

Notes

- α controls importance of recent history
- Commonly α = 0.5
- SJF preemptive version is called **Shortest-Remaining-Time-First (SRTF)**

Numerical Example – Exponential Averaging

Given:

- $T_1 = 10$
- $\alpha = 0.5$
- Previous CPU bursts: **8, 7, 4, 16**

Since SJF is non-preemptive, order becomes: **4, 7, 8, 16**

Calculations

- $T_2 = 0.5 \times 4 + 0.5 \times 10 = 7$
- $T_3 = 0.5 \times 7 + 0.5 \times 7 = 7$
- $T_4 = 0.5 \times 8 + 0.5 \times 7 = 7.5$
- $T_5 = 0.5 \times 16 + 0.5 \times 7.5 = 11.8$

Result

- **Predicted 5th CPU burst = 11.8**

Prediction Behavior with Different α Values

Special Cases

- α = 0
    - $\tau_{n+1} = \tau_n$
    - Recent burst ignored
- α = 1
    - $\tau_{n+1} = t_n$
    - Only last burst considered

Expanded Formula

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\alpha t_{n-1} + \cdots + (1-\alpha)^{n+1}\tau_0$$

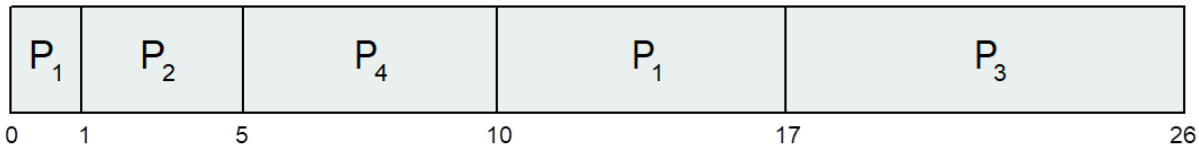
Each older burst has **less weight**, ensuring stability.

Shortest-Remaining-Time-First (SRTF)

- **Preemptive version of SJF**
- If a new process arrives with **shorter remaining burst time**, it preempts the running process

SRTF Example

Process	*Arrival* Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5



Average Waiting Time

$$[(10-1) + (1-1) + (17-2) + (5-3)]/4 = 6.5 \text{ ms}$$